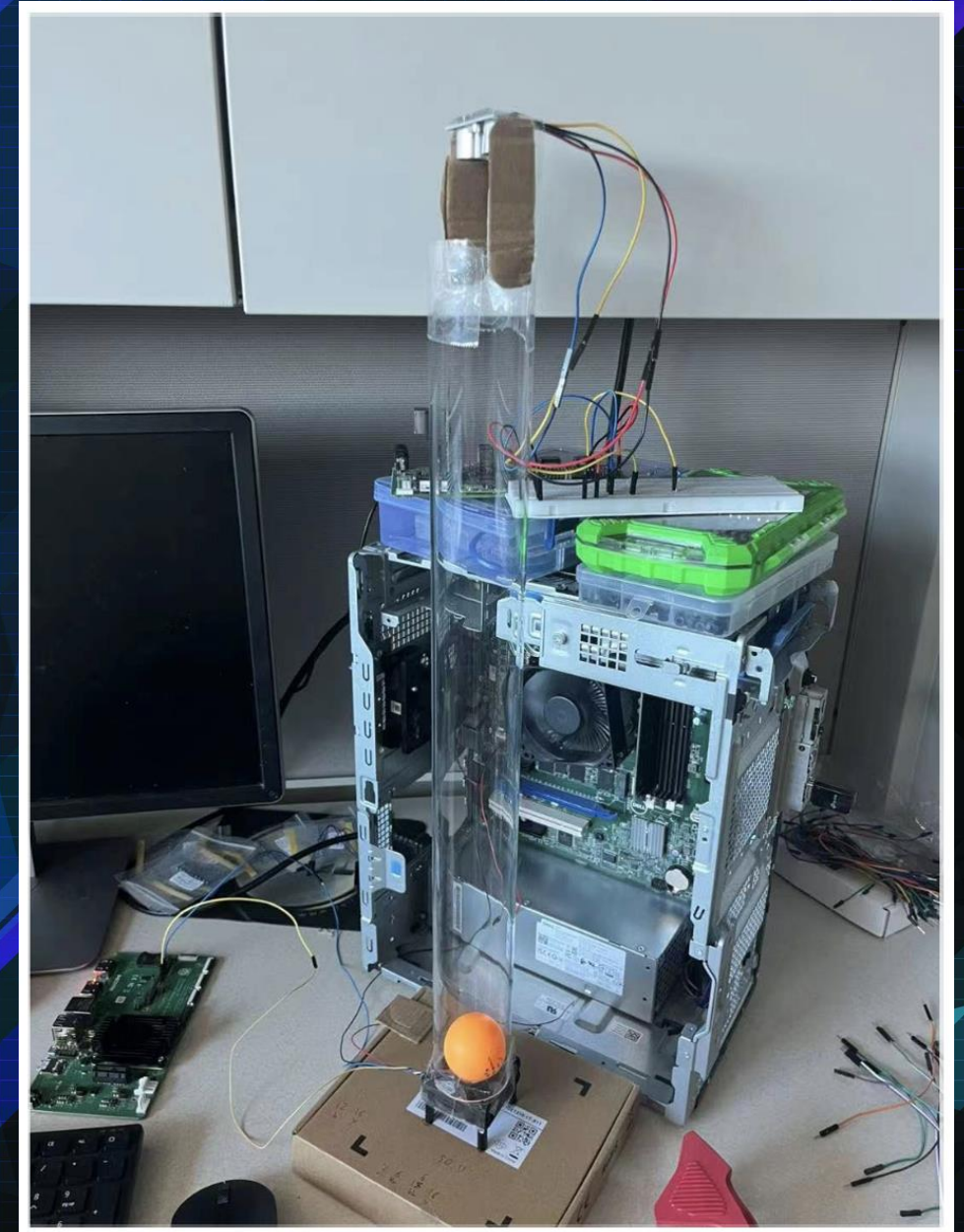

“Experimental Performance Evaluation of Real-Time Traffic Scheduler in Control Systems”

Final Progress Report

SE5402/CSE5312: ARCHITECTURE OF
INTERNET OF THINGS

FALL 2025

ABBY HORNING & JAKE THURMAN



Project Recap

Overall Goal

- The primary goal is to evaluate the effect of traffic scheduling on a real-time system under heavy network congestion.

Objectives

- Understanding the inner workings of the Linux networking stack.
- Developing and applying a PID controller for system stabilization.
- Gaining familiarity with real-time scheduling mechanisms.
- Building practical skills in both C and Python programming.

Project Management: Timeline & Roles



Phase 1 — System Foundation

- Establish reliable Wi-Fi communication between Sensor Pi and Fan Pi
- Implement and tune the PID control loop



Phase 2 — Congestion Injection

- Add configurable delay/jitter/loss
- Observe degraded control performance



Phase 3 — Real-Time Scheduler Integration

- Apply Linux tc-based prioritization (ETF-style traffic)
- Improve timing and control responsiveness

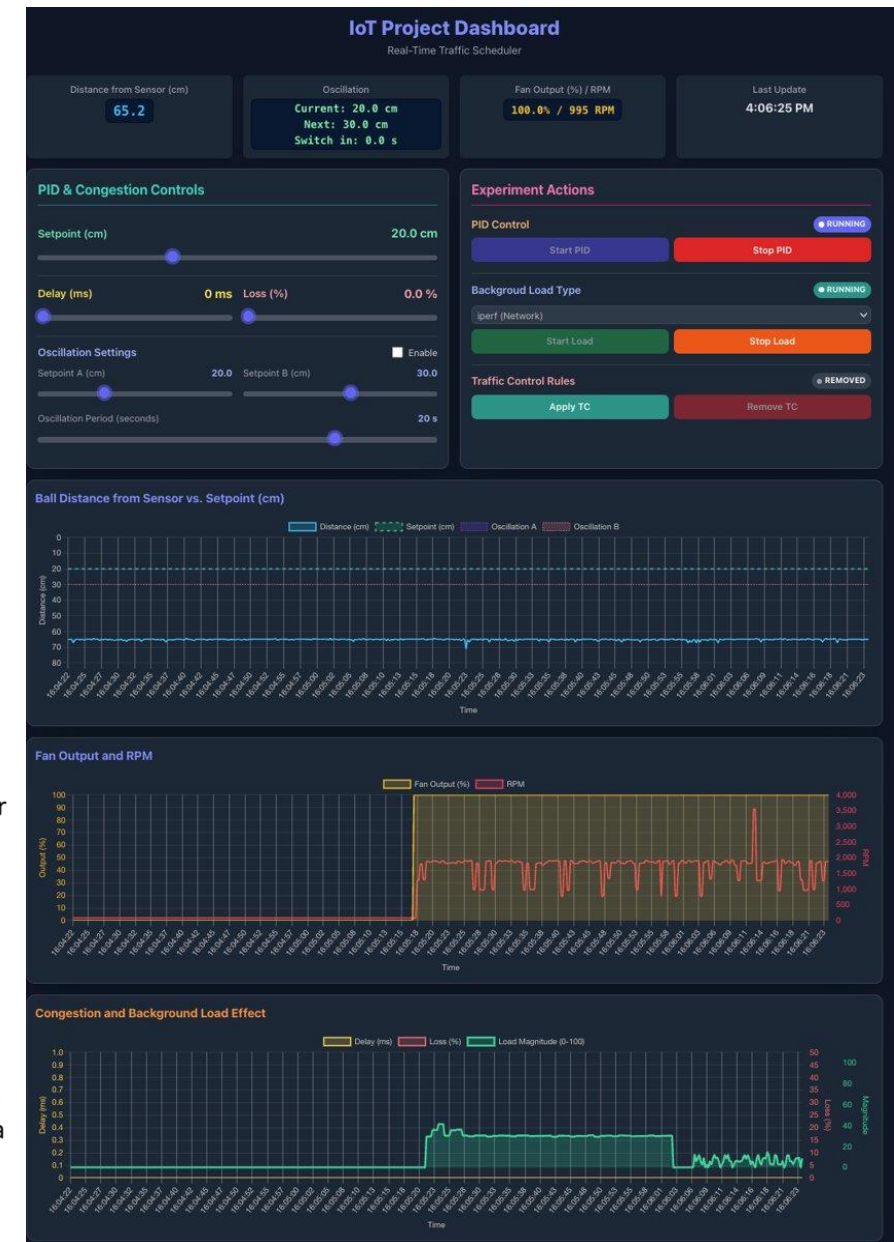
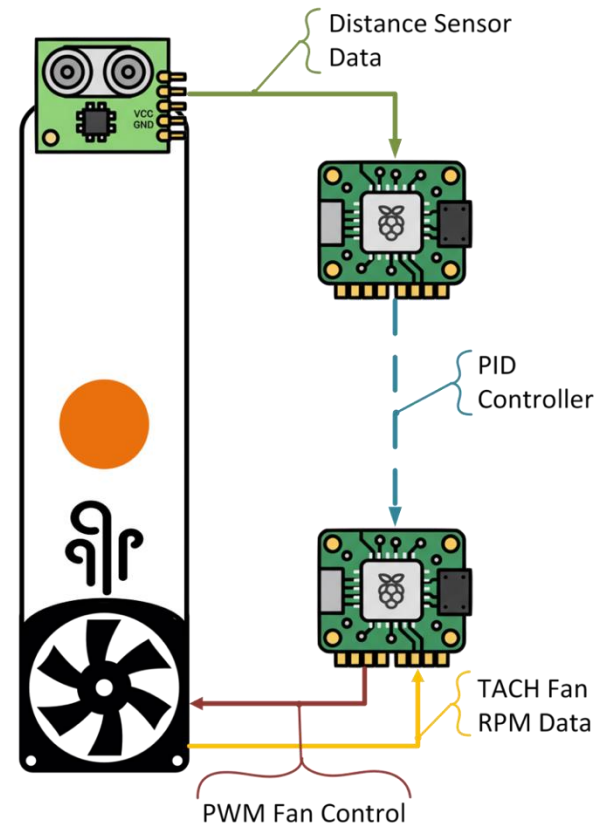


Phase 4 — Evaluation & Deliverables

- Run full oscillation experiments
- Log and analyze height, error, overshoot %, and rise and fall times
- Complete final report and presentation

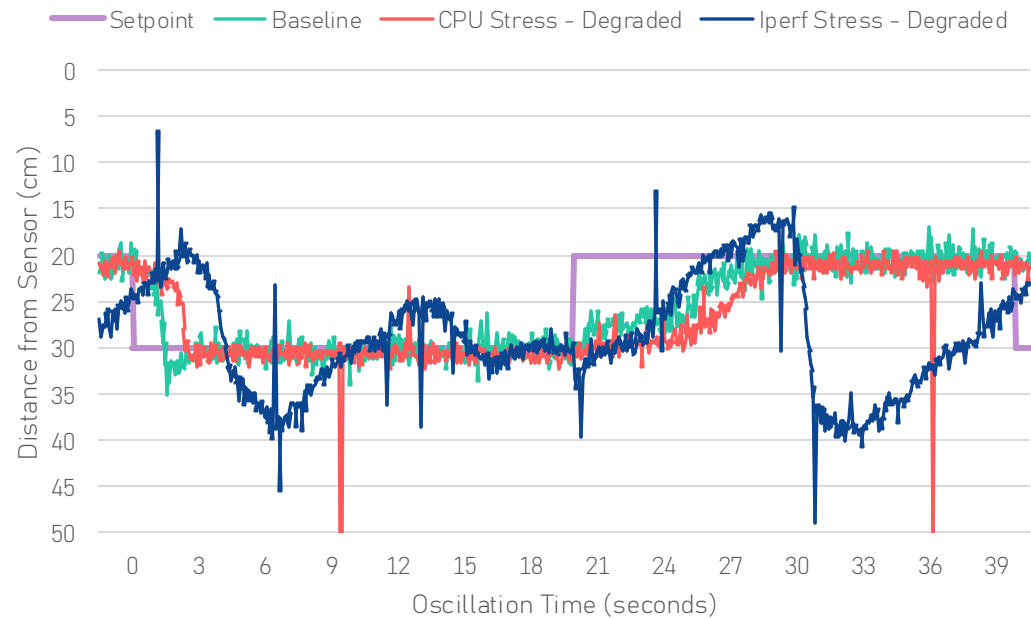
System Architecture

- Sensor Node
 - Ultrasonic sensor
 - PID control loop
 - Web app & telemetry collector
- Fan Controller Node
 - Adjusts fan PWM via EMC2301
- Dashboard / Experiment Manager
 - Starts/stops oscillation and experiments
 - Injects congestion (delay, jitter, loss)
 - Start the traffic controller
 - Displays real-time telemetry
 - Central control for the entire system

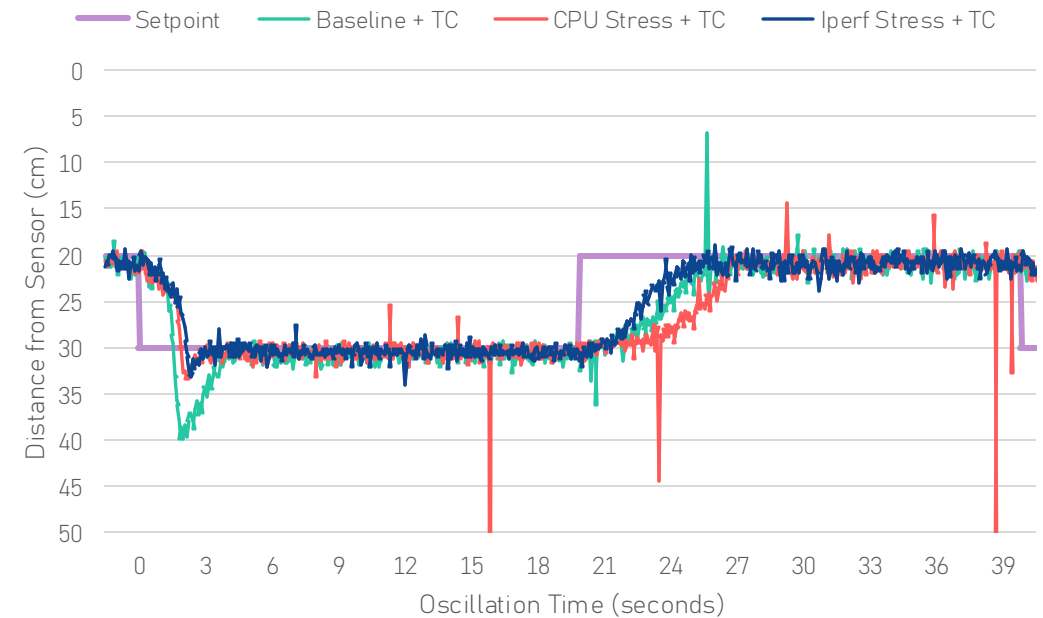


Testing & Performance Overview

BASELINE VS DEGRADED



BASELINE VS TC IMPROVED



$$\sum_{t=0}^T |SP(t) - PV(t)| \cdot \Delta t$$

Integral Absolute Error (IAE)

Measures: Total accumulated tracking error

Shows: Overall Accuracy

Target Result: Lower IAE when TC rules are applied.

$$\sum_{t=0}^T (SP(t) - PV(t))^2 \cdot \Delta t$$

Integral Squared Error (ISE)

Measures: Error energy with large deviations heavily penalized

Shows: Stability and Peak Deviation (quantifies instability).

Target Result: Dramatically reduced ISE when TC rules are applied.

$$((P_{v_{peak}} - SP_{new}) / SP_{new}) \cdot 100$$

Overshoot Percentage

Measures: Maximum peak above new setpoint during transitions

Shows: Aggressiveness/Responsiveness.

Target Result: Lower overshoot and smoother control responses.

Rise & Fall Time

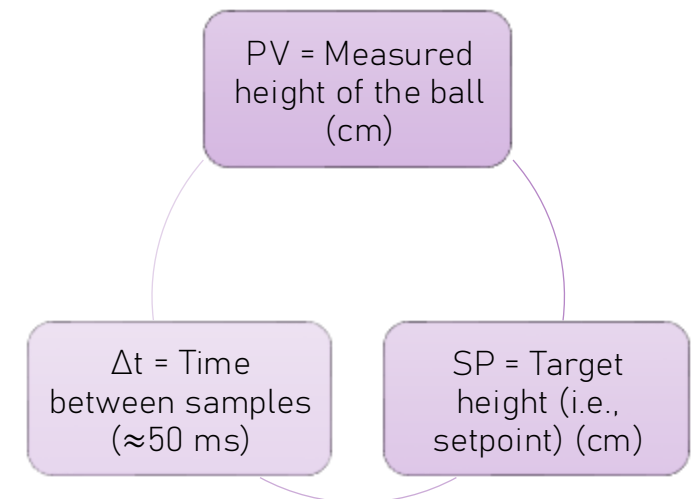
e.g.: *Settling Time*

Measures: Time for ball to move from 10% → 90% of setpoint change

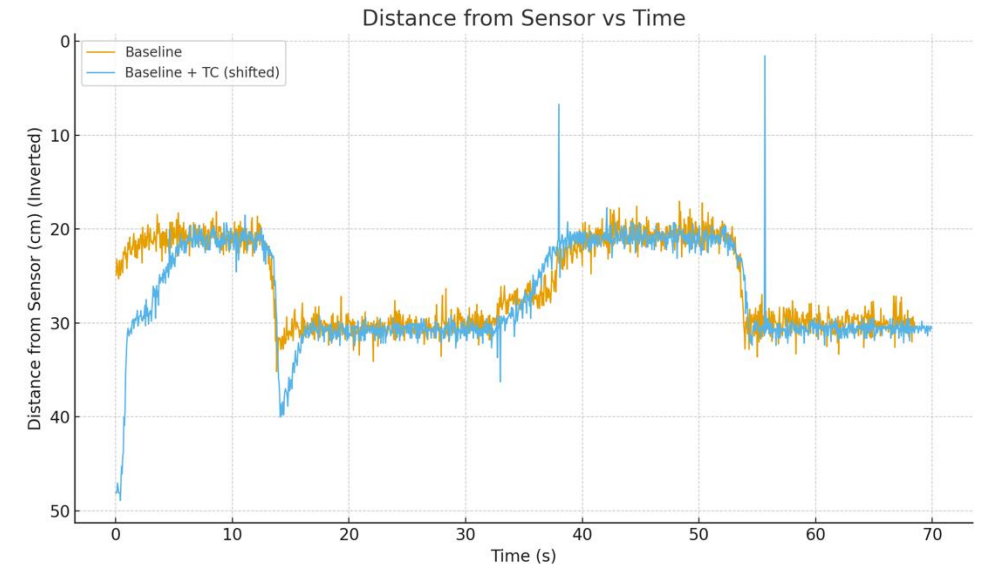
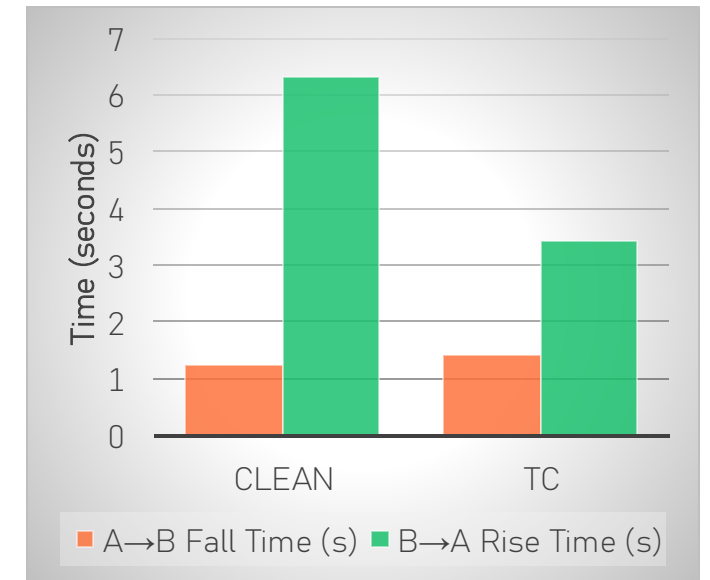
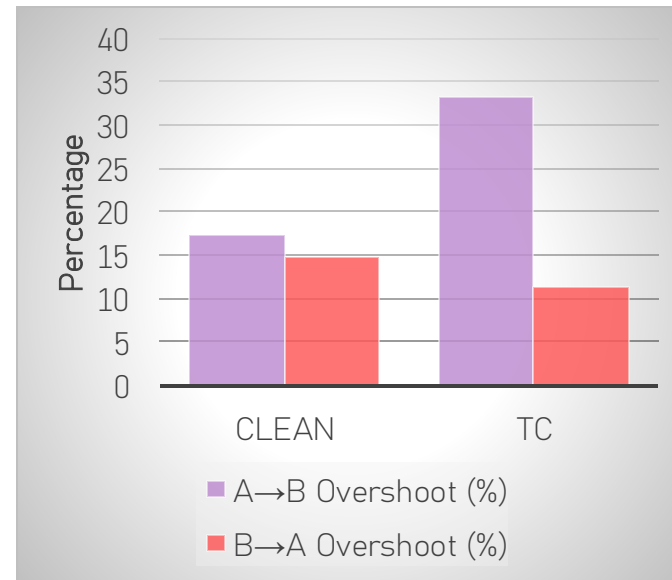
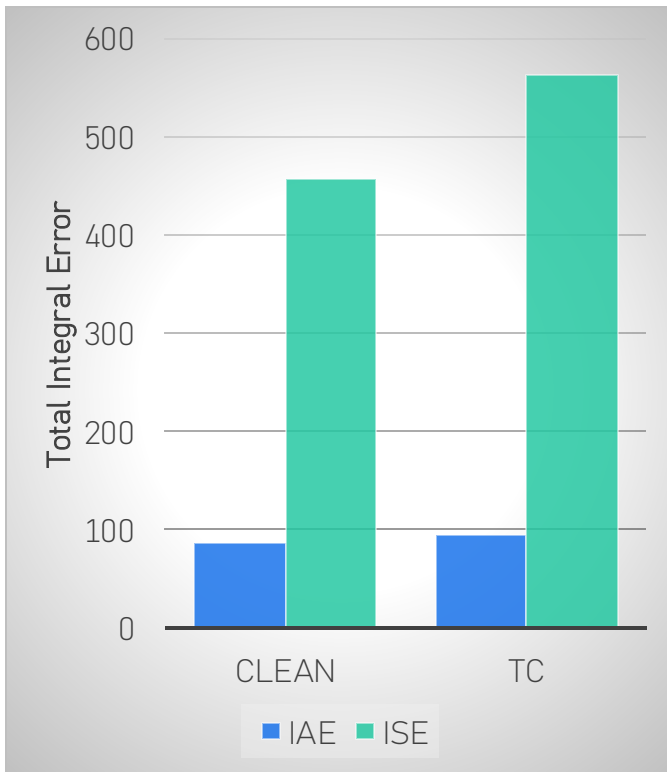
Shows: Speed/Latency

Target Result: Significant decrease in rise time with TC rules applied.

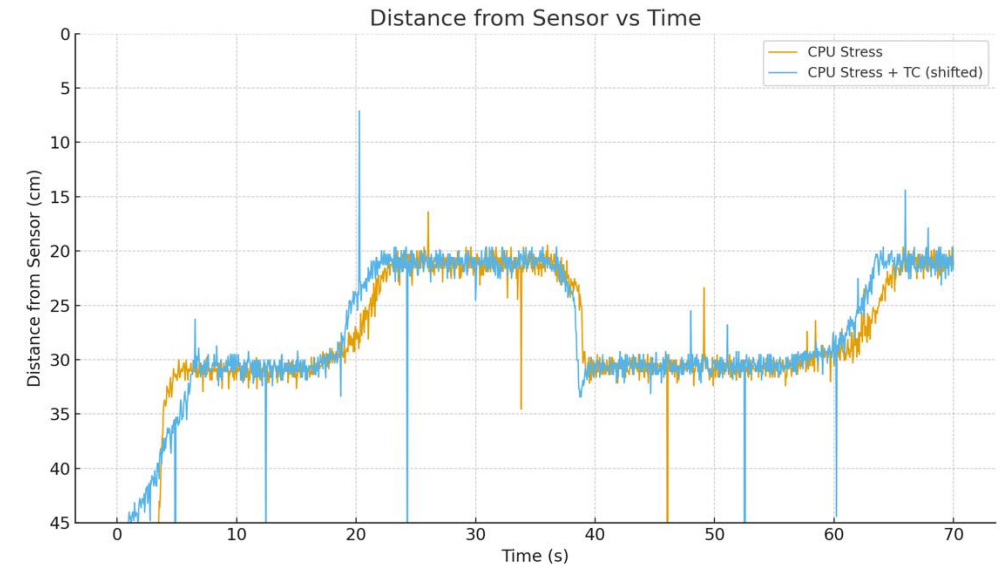
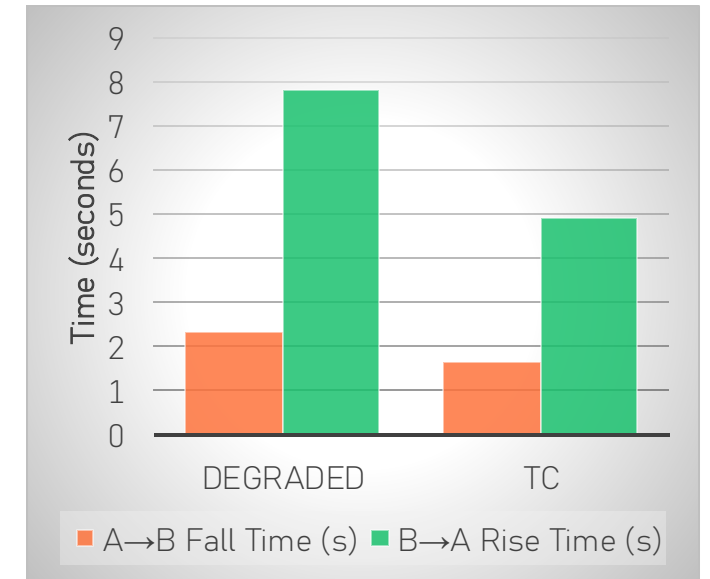
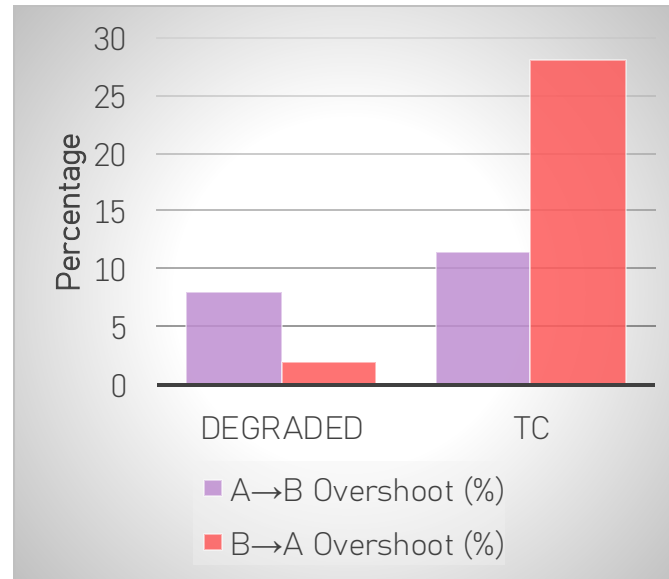
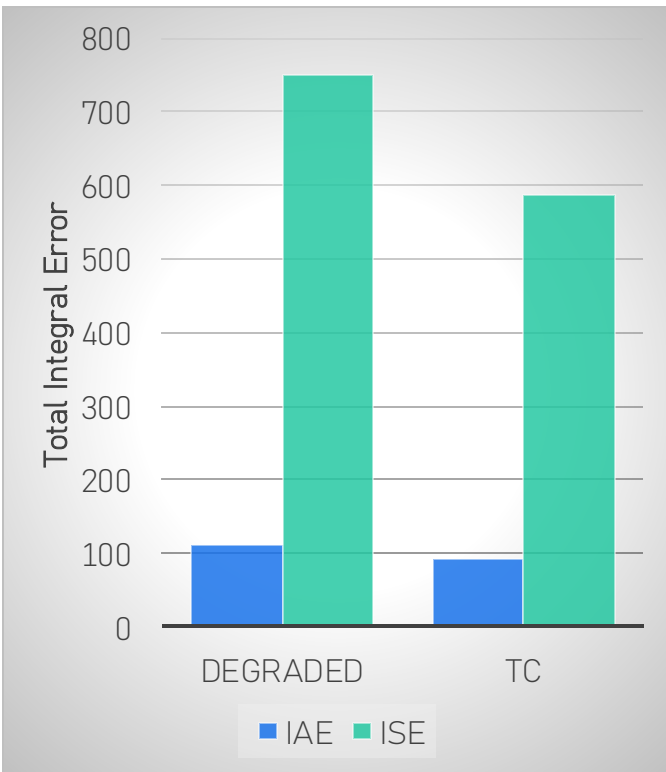
Performance Evaluation Metrics



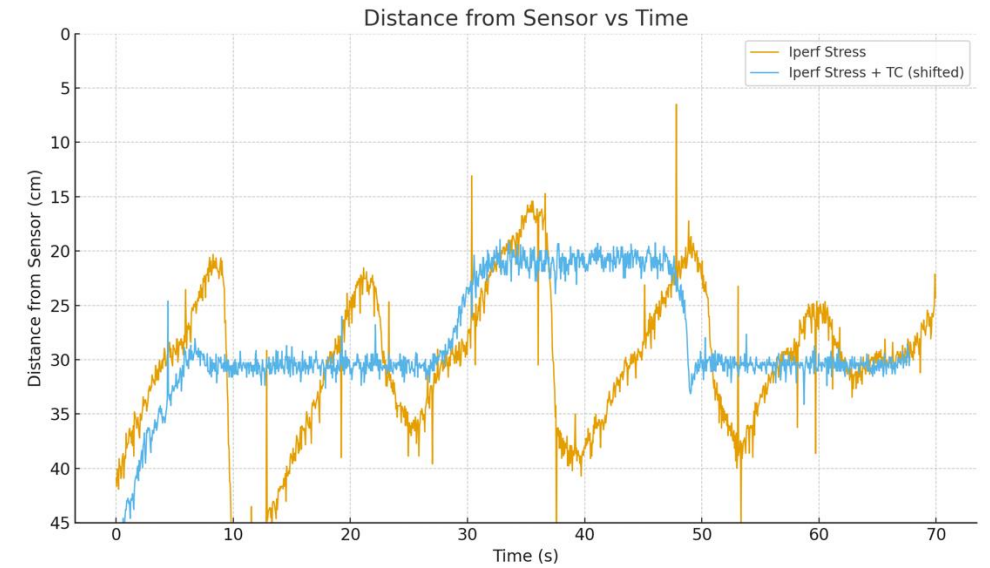
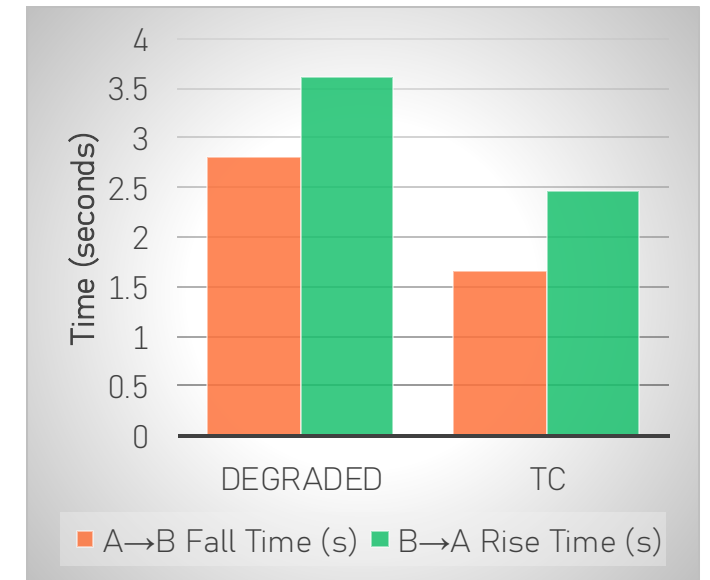
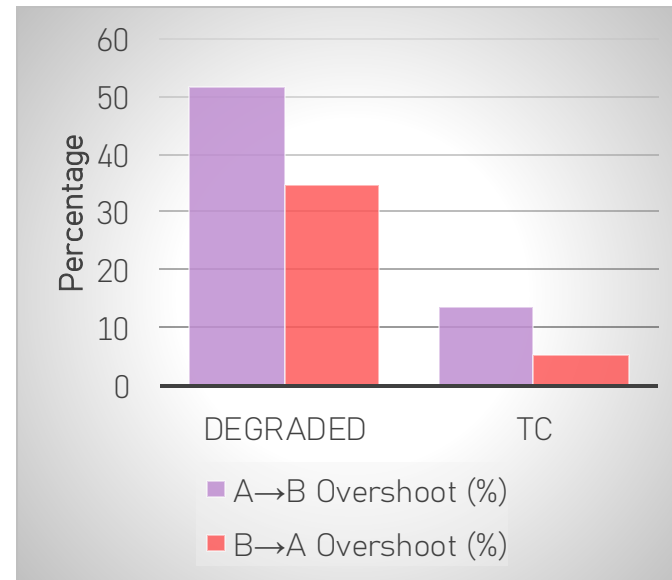
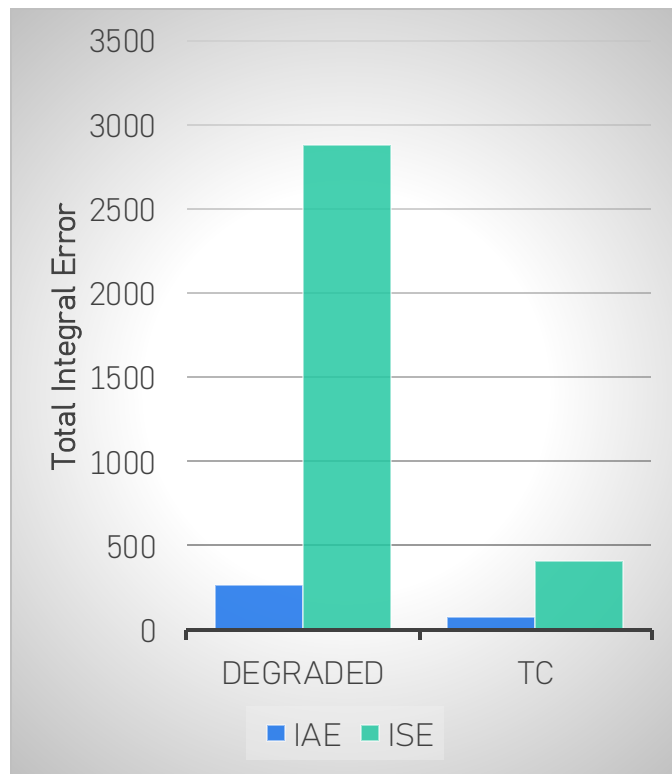
Baseline: Clean vs TC Improved



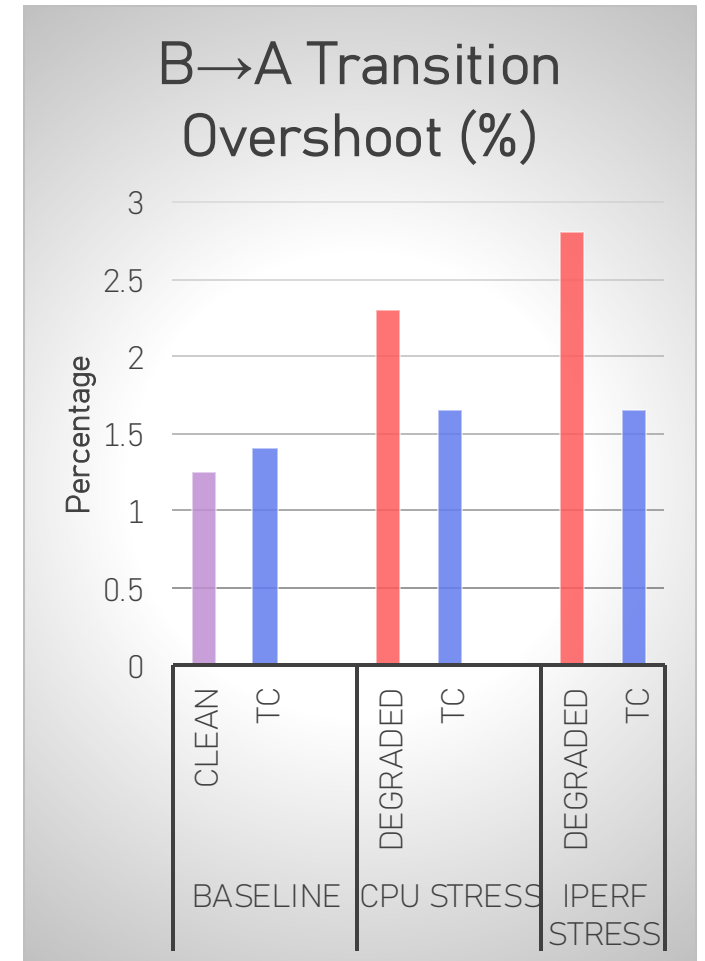
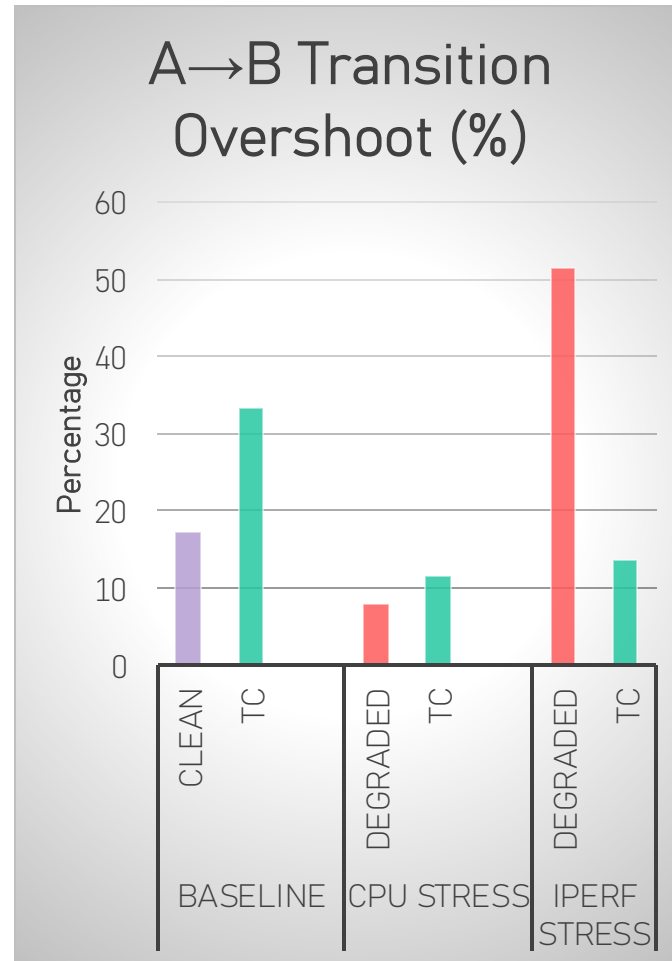
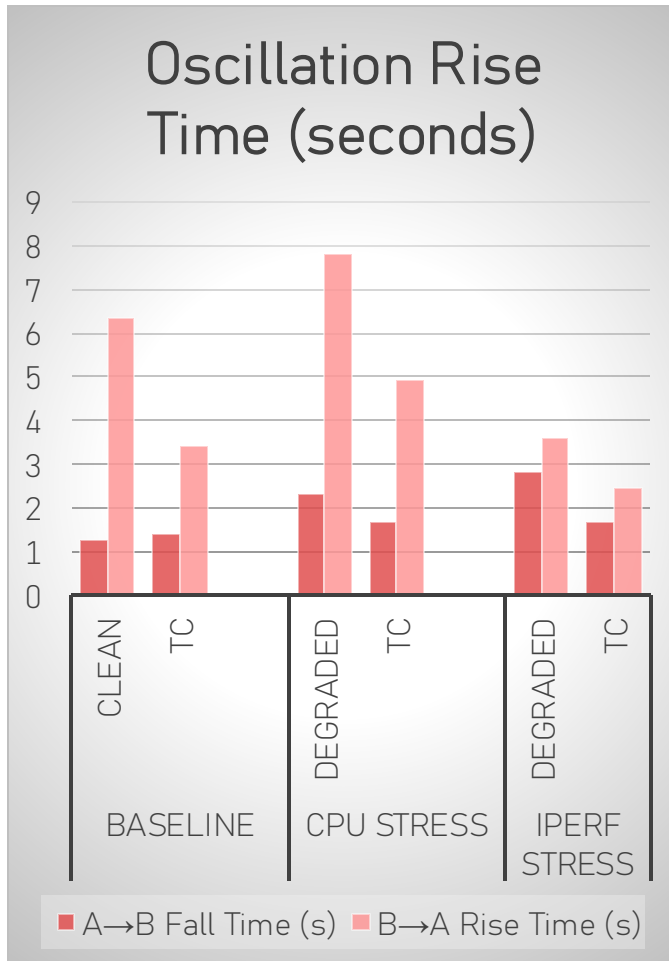
CPU Stress: Degraded vs TC Improved



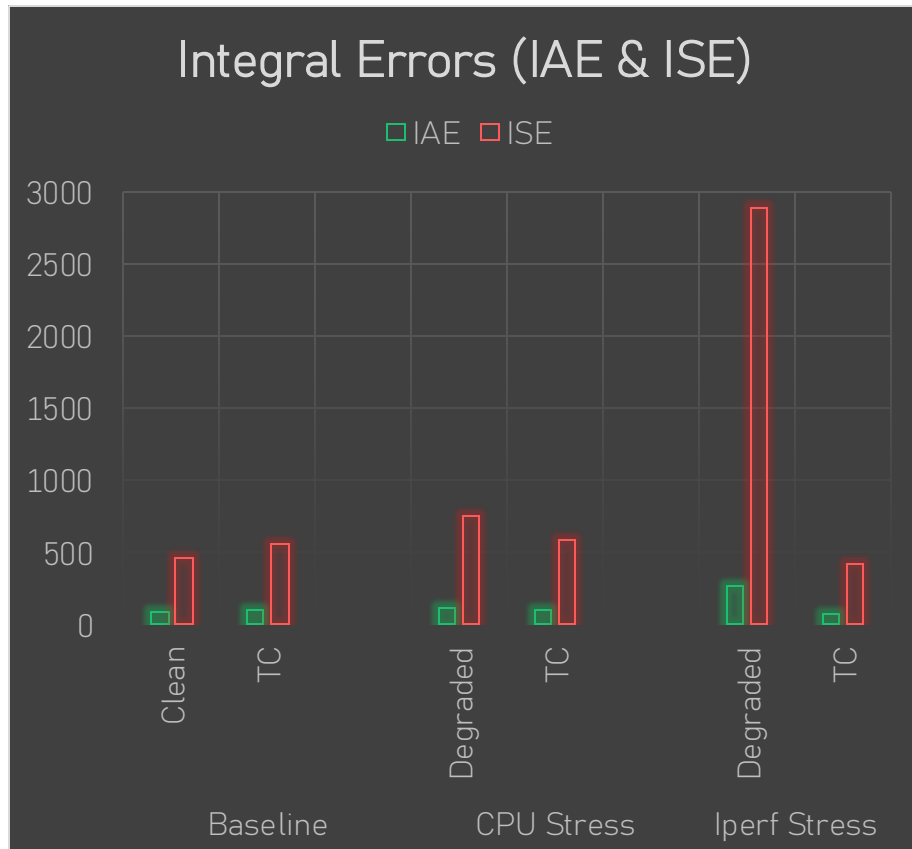
Iperf (Network Stress): Degraded vs TC Improved



Metric Comparisons



Results Summary



Baseline

- Controller performs smoothly with minimal error
- TC scheduling has little effect under ideal conditions
- Slight increase in overshoot with TC, small improvement on downward transitions

CPU Stress (Actuator Node Loaded)

- CPU load introduces scheduler jitter, slowing controller reactions
- TC scheduling reduces timing jitter, improving:
 - Tracking error (IAE: 112 → 96)
 - Transition response times
- Some overshoot increases, but overall stability improves

Iperf Stress (Heavy Network Congestion)

- Severe network jitter causes large errors, delay, and unstable tracking
- TC scheduling provides the largest improvement:
 - IAE reduced 259 → 74
 - ISE reduced 2880 → 411
 - Overshoot drastically reduced for both transitions
 - Faster, smoother rise/fall times

Key Takeaways



Timing determinism is critical for distributed real-time control



CPU stress mainly affects control through scheduler delays



Network congestion is the most damaging interference source



TC scheduling is highly effective, especially under heavy network load



Prioritizing control traffic restores stability and greatly reduces error



References

- Salzmann et al. (2025): Hovering a ping-pong ball: A demonstration setup for teaching PID control
 - (<https://doi.org/10.26434/chemrxiv-2025-328tk>)
- An example of how Linux traffic scheduler improves control under traffic congestion:
 - (<https://github.com/NXP/dds-tsn>)
- Depth/Distance Sensors:
 - (<https://randomnerdtutorials.com/complete-guide-for-ultrasonic-sensor-hc-sr04/>)
- Tuning PID parameters:
 - https://davidr.no/iav3017/papers/Ziegler_Nichols_%201942.pdf
- PWM Fan Control:
 - PWM basics: <https://www.arduino.cc/en/Tutorial/Foundations/PWM>
 - PWM fan control: <https://github.com/folkhack/raspberry-pi-pwm-fan-2>

<https://github.com/jthurm11/iot-real-time-scheduler-evaluation>

